Lab Report

Only for Course Teacher						
		Needs Improvement	Developing	Sufficient	Above Average	Total Mark
Allocate mark & Percentage		25%	50%	75%	100%	15
Problem Analysis	03					
Solution Design	02					
Code Development	06					
Accuracy	04					
					Total obtained mark	
Comments						

Semester: Spring 2026

Student Name: MD. Ekramul Basher Tashin

Student ID: 242-35-504

Batch: 43 **Section: L1**

Course Code: SE215 **Course Name: Algorithm Design & Analysis Lab**

Course Teacher Name: Syed Shams Islam

Designation: Lecturer

Submission Date: 19-04-2026

Algorithm Design & Analysis

Lab Report

MD.Ekramul Basher Tasher

ID: 242-35-504

Section: 43L1

Problem-1: Median via Selection Sort Invariant

Code:

```
#include <stdio.h>
int find_median(int *arr, int size) {
    if (size == 0) return -1;
    int i, j, min_idx, temp;
    int mid = size / 2;
    for (i = 0; i <= mid; i++) {
        min_idx = i;
        for (j = i + 1; j < size; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }
        temp = arr[i];
        arr[i] = arr[min_idx];
        arr[min_idx] = temp;
    }
    return arr[mid];
```

```
}
int main() {
    int arr[] = {7, 2, 10, 9, 1};
    int size = 5;
    int median = find_median(arr, size);
    printf("%d\n", median);
    return 0;
}
```

Output

```
7


=== Code Execution Successful ===
```

Problem-2: K-th Largest via Bubble Sort Invariant

Code:

```
#include <stdio.h>
int find_kth_largest(int *arr, int size, int k) {
    int i, j, temp;
    for (i = 0; i < k; i++) {
        for (j = 0; j < size - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
```

```
            arr[j + 1] = temp;
        }
    }
  }
  return arr[size - k];
}
int main() {
  int arr[] = {3, 1, 7, 4, 9, 2};
  int size = 6, k = 2;

  printf("%d\n", find_kth_largest(arr, size, k));
  return 0;
}
```

Output

```
7

=== Code Execution Successful ===
```

Problem-3: Merge Two Arrays via Insertion Sort Logic

Code:

```
#include <stdio.h>
void merge_sorted(int *a, int n, int *b, int m, int *out) {
  int i, j, key;
```

```
    for (i = 0; i < n + m; i++) {
        if (i < n)
            key = a[i];
        else
            key = b[i - n];
        j = i - 1;
        while (j >= 0 && out[j] > key) {
            out[j + 1] = out[j];
            j--;
        }
        out[j + 1] = key;
    }
}
int main() {
    int a[] = {5, 1, 9};
    int b[] = {3, 7, 2, 6};
    int n = 3, m = 4;
    int out[7];
    merge_sorted(a, n, b, m, out);
    for (int i = 0; i < n + m; i++) {
        printf("%d ", out[i]);
    }
    return 0;
}
```

```
Output

1 2 3 5 6 7 9

=== Code Execution Successful ===
```

Problem-4.1: Factorial

Code:

```
#include <stdio.h>
long long factorial(int n) {
    long long result = 1;
    int i;
    for (i = 1; i <= n; i++) {
        result = result * i;
    }
    return result;
}
int main() {
    int n = 5;
    printf("%lld\n", factorial(n));
    return 0;
}
```

```
Output

120


=== Code Execution Successful ===
```

Problem-4.2: Fibonacci

Code:

```
#include <stdio.h>
long long fibonacci(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fibonacci(n - 1) + fibonacci(n - 2);
}
int main() {
    int n = 10;
    printf("%lld\n", fibonacci(n));
    return 0;
}
```

```
Output

55


=== Code Execution Successful ===
```

Problem-4.3: Count Digits

Code:

```
#include <stdio.h>
int count_digits(int n) {
    if (n == 0) return 1;
    if (n < 10) return 1;
    return 1 + count_digits(n / 10);
}
int main() {
    int n = 98765;
    printf("%d\n", count_digits(n));
    return 0;
}
```

Output

```
5


=== Code Execution Successful ===
```

Problem-4.4: Power Function

Code:

```
#include <stdio.h>
double power(double base, int exp) {
    double result = 1.0;
    int i;

    for (i = 0; i < exp; i++) {
        result *= base;
    }
```

```
    return result;
}
int main() {
    double base = 2.5;
    int exp = 3;

    printf("%lf\n", power(base, exp));
    return 0;
}
```

Output

```
15.625000


=== Code Execution Successful ===
```

Problem-4.5: Palindrome Check

Code:

```
#include <stdio.h>
#include <string.h>
int is_palindrome(char *str, int left, int right) {
    if (left >= right) return 1;
    if (str[left] != str[right]) return 0;
    return is_palindrome(str, left + 1, right - 1);
}
int main() {
    char str[] = "racecar";
    int len = strlen(str);
    printf("%d\n", is_palindrome(str, 0, len - 1));
```

```
    return 0;
}
```

Output

```
1


=== Code Execution Successful ===
```

<u>Problem-4.6: Sum of Array</u>

Code:

```
#include <stdio.h>
int array_sum(int *arr, int size) {
    if (size == 0) return 0;
    return arr[size - 1] + array_sum(arr, size - 1);
}
int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int size = 5;
    printf("%d\n", array_sum(arr, size));
    return 0;
}
```

Output

```
15


=== Code Execution Successful ===
```

Problem-4.7: Maximum of Array

Code:

```
#include <stdio.h>
int array_max(int *arr, int size) {
    if (size == 1) return arr[0];
    int max = array_max(arr, size - 1);
    if (arr[size - 1] > max) return arr[size - 1];
    return max;
}
int main() {
    int arr[] = {3, 1, 9, 2, 7};
    int size = 5;
    printf("%d\n", array_max(arr, size));
    return 0;
}
```

Output

```
9


=== Code Execution Successful ===
```

Problem-4.8: Binary Search

Code:

```
#include <stdio.h>
int binary_search(int *arr, int left, int right, int target)
{
    if (left > right)
        return -1;
    int mid = left + (right - left) / 2;
    if (arr[mid] == target)
        return mid;
    else if (arr[mid] > target)
        return binary_search(arr, left, mid - 1, target);
    else
        return binary_search(arr, mid + 1, right, target);
}
int main()
{
    int arr[] = {1,3,5,7,9};
    int n = 5;
    int target = 7;
    int result = binary_search(arr, 0, n - 1, target);
    printf("%d\n", result);
    return 0;
}
```

Output

```
3


=== Code Execution Successful ===
```

Problem-5: Merge sort

Code:

```
#include <stdio.h>
void merge(int *arr, int left, int mid, int right)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int i = 0; i < n2; i++)
        R[i] = arr[mid + 1 + i];
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }
    while (i < n1)
        arr[k++] = L[i++];
    while (j < n2)
        arr[k++] = R[j++];
}
void merge_sort(int *arr, int left, int right)
{
    if (left < right)
    {
        int mid = left + (right - left) / 2;
        merge_sort(arr, left, mid);
        merge_sort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
```

```
}
int main()
{
    int arr[] = {38, 27, 43, 3, 9, 82, 10};
    int n = 7;
    merge_sort(arr, 0, n - 1);
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

Output

```
3 9 10 27 38 43 82

=== Code Execution Successful ===
```

<u>Problen-6: Quick sort</u>

Code:

```
#include <stdio.h>
int swap_count = 0;
void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
    swap_count++;
}
int partition(int *arr, int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
```

```
    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}
void quick_sort(int *arr, int low, int high)
{
    if (low < high)
    {
        int pi = partition(arr, low, high);
        quick_sort(arr, low, pi - 1);
        quick_sort(arr, pi + 1, high);
    }
}
int main()
{
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = 6;
    quick_sort(arr, 0, n - 1);
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\nSwaps: %d\n", swap_count);
    return 0;
}
```

```
Output

1 5 7 8 9 10
Swaps: 5


=== Code Execution Successful ===
```

Problem-7: Fractional Knapsack

Code:

```
#include <stdio.h>
#include <stdlib.h>
typedef struct {
    int weight;
    int value;
} Item;
int compare(const void *a, const void *b)
{
    double r1 = (double)((Item *)b)->value / ((Item *)b)->weight;
    double r2 = (double)((Item *)a)->value / ((Item *)a)->weight;
    if (r1 > r2) return 1;
    else if (r1 < r2) return -1;
    return 0;
}
double fractional_knapsack(Item *items, int n, int capacity)
{
    qsort(items, n, sizeof(Item), compare);
    double total = 0.0;
    for (int i = 0; i < n; i++)
    {
```

```
        if (capacity >= items[i].weight)
        {
            capacity -= items[i].weight;
            total += items[i].value;
        }
        else
        {
            total += (double)items[i].value * capacity / items[i].weight;
            break;
        }
    }
    return total;
}
int main()
{
    Item items[] = {{10,60},{20,100},{30,120}};
    int n = 3;
    int capacity = 50;
    double result = fractional_knapsack(items, n, capacity);
    printf("%.2lf\n", result);
    return 0;
}
```

Output

```
240.00


=== Code Execution Successful ===
```

Problem-8: Minimum Spanning Tree

Code:

```
#include <stdio.h>
#include <limits.h>
void prims(int g[5][5], int V)
{
    int key[5], parent[5], used[5];
    for(int i=0; i<V; i++)
    {
        key[i] = INT_MAX;
        used[i] = 0;
    }
    key[0] = 0;
    parent[0] = -1;
    for(int i=0; i<V-1; i++)
    {
        int min = INT_MAX, u;
        for(int j=0; j<V; j++)
        {
            if(!used[j] && key[j] < min)
            {
                min = key[j];
                u = j;
            }
        }
        used[u] = 1;
        for(int v=0; v<V; v++)
        {
            if(g[u][v] && !used[v] && g[u][v] < key[v])
            {
                key[v] = g[u][v];
                parent[v] = u;
            }
        }
```

```
    }
    int total = 0;
    printf("Prim:\n");
    for(int i=1; i<V; i++)
    {
        printf("%d-%d %d\n", parent[i], i, g[i][parent[i]]);
        total += g[i][parent[i]];
    }
    printf("Total %d\n", total);
}
int main()
{
    int g[5][5] = {
        {0,2,0,6,0},
        {2,0,3,8,5},
        {0,3,0,0,7},
        {6,8,0,0,9},
        {0,5,7,9,0}
    };
    prims(g,5);
    return 0;
}
```

```
Output

Prim:
0-1 2
1-2 3
0-3 6
1-4 5
Total 16


=== Code Execution Successful ===
```

<u>Problem-9a: Dijkstra's Algorithm</u>

Code:

```
#include <stdio.h>
#include <limits.h>
int minDist(int dist[], int used[], int V)
{
    int min = INT_MAX, idx;
    for(int i = 0; i < V; i++)
        if(!used[i] && dist[i] < min)
            min = dist[i], idx = i;
    return idx;
}
void dijkstra(int g[5][5], int src, int V)
{
    int dist[5], used[5];
    for(int i = 0; i < V; i++)
        dist[i] = INT_MAX, used[i] = 0;
```

```
    dist[src] = 0;
    for(int i = 0; i < V - 1; i++)
    {
        int u = minDist(dist, used, V);
        used[u] = 1;
        for(int v = 0; v < V; v++)
            if(g[u][v] && !used[v] && dist[u] != INT_MAX && dist[u] + g[u][v] <
dist[v])
                dist[v] = dist[u] + g[u][v];
    }
    printf("Vertex Distance\n");
    for(int i = 0; i < V; i++)
        printf("%d %d\n", i, dist[i]);
}
int main()
{
    int g[5][5] = {
        {0,4,0,0,8},
        {4,0,8,0,11},
        {0,8,0,7,0},
        {0,0,7,0,9},
        {8,11,0,9,0}
    };
    dijkstra(g, 0, 5);
    return 0;
}
```

```
Output

Vertex Distance
0  0
1  4
2  12
3  17
4  8


=== Code Execution Successful ===
```

Problem-9b: Bellman-Ford Algorithm

Code:

```
#include <stdio.h>
#include <limits.h>
typedef struct {
    int u, v, w;
} Edge;
void bellman_ford(int V, int E, Edge *edges, int src)
{
    int dist[10];
    for(int i = 0; i < V; i++)
        dist[i] = INT_MAX / 2;
    dist[src] = 0;
    for(int i = 1; i <= V - 1; i++)
        for(int j = 0; j < E; j++)
            if(dist[edges[j].u] + edges[j].w < dist[edges[j].v])
                dist[edges[j].v] = dist[edges[j].u] + edges[j].w;
    for(int j = 0; j < E; j++)
```

```
        if(dist[edges[j].u] + edges[j].w < dist[edges[j].v])
        {
            printf("Graph contains a negative-weight cycle\n");
            return;
        }
    printf("Vertex Distance\n");
    for(int i = 0; i < V; i++)
        printf("%d %d\n", i, dist[i]);
}
int main()
{
    Edge edges[] = {
        {0,1,6},{0,2,7},{1,2,8},{1,3,5},
        {1,4,-4},{2,3,-3},{2,4,9},{3,1,-2},
        {4,0,2},{4,3,7}
    };
    bellman_ford(5, 10, edges, 0);
    return 0;
}
```

Output

```
Vertex Distance
0 0
1 2
2 7
3 4
4 -2


=== Code Execution Successful ===
```

Problem-10a: 0/1 Knapsack

Code:

```
#include <stdio.h>
int knapsack_01(int *wt, int *val, int n, int W)
{
    int dp[n+1][W+1];
    for(int i=0;i<=n;i++)
        for(int w=0;w<=W;w++)
            if(i==0 || w==0) dp[i][w]=0;
            else if(wt[i-1]<=w)
                dp[i][w] = dp[i-1][w] > dp[i-1][w-wt[i-1]] + val[i-1] ? dp[i-1][w] :
dp[i-1][w-wt[i-1]] + val[i-1];
            else dp[i][w]=dp[i-1][w];
    return dp[n][W];
}
int main()
{
    int wt[] = {2,3,4,5};
    int val[] = {3,4,5,6};
    int n = 4;
    int W = 5;
    printf("%d\n", knapsack_01(wt,val,n,W));
    return 0;
}
```

Output

```
7


=== Code Execution Successful ===
```

Problem-10b: Minimum Number of Coins

Code:

```
#include <stdio.h>
#include <limits.h>
int min_coins(int *coins, int n, int amount)
{
    int dp[amount+1];
    for(int i=0;i<=amount;i++)
        dp[i] = INT_MAX-1;
    dp[0] = 0;
    for(int i=1;i<=amount;i++)
        for(int j=0;j<n;j++)
            if(coins[j]<=i && dp[i-coins[j]]+1 < dp[i])
                dp[i] = dp[i-coins[j]] + 1;
    return dp[amount]==INT_MAX-1 ? -1 : dp[amount];
}
int main()
{
    int coins[] = {1,5,6,9};
    int n = 4;
    int amount = 11;
    printf("%d\n", min_coins(coins,n,amount));
    return 0;
}
```

Output

```
2


=== Code Execution Successful ===
```

Problem-10c: Minimum Ways of Coin Change (Count of Combinations)

Code:

```
#include <stdio.h>
int count_ways(int *coins, int n, int amount)
{
    int dp[amount+1];
    for(int i=0;i<=amount;i++) dp[i]=0;
    dp[0]=1;
    for(int i=0;i<n;i++)
        for(int j=coins[i];j<=amount;j++)
            dp[j] += dp[j-coins[i]];
    return dp[amount];
}
int main()
{
    int coins[] = {1,2,3};
    int n = 3;
    int amount = 4;
    printf("%d\n", count_ways(coins,n,amount));
    return 0;
}
```

Output

```
4


=== Code Execution Successful ===
```

Problem-10d: Longest Common Subsequence (LCS)

Code:

```
#include <stdio.h>
#include <string.h>
int max(int a,int b){ return a>b?a:b; }
int lcs(char *s1, char *s2, int m, int n)
{
    int dp[m+1][n+1];
    for(int i=0;i<=m;i++)
        for(int j=0;j<=n;j++)
            if(i==0 || j==0) dp[i][j]=0;
            else if(s1[i-1]==s2[j-1]) dp[i][j]=dp[i-1][j-1]+1;
            else dp[i][j]=max(dp[i-1][j], dp[i][j-1]);
    return dp[m][n];
}
int main()
{
    char s1[]="ABCBDAB";
    char s2[]="BDCAB";
    printf("%d\n", lcs(s1,s2,strlen(s1),strlen(s2)));
    return 0;
}
```

Output

```
4


=== Code Execution Successful ===
```